

ASSIGNMENT-11

NAME:E.NIKHIL

H.NO:2303A51847

B-13

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack: pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Prompt:

Write a Python Stack class with push, pop, peek, and is_empty methods.

Code:

```
#Write a Python Stack class with push, pop, peek, and is_empty methods.
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("peek from empty stack")

    def is_empty(self):
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek()) # Output: 3
    print(stack.pop()) # Output: 3
    print(stack.is_empty()) # Output: False
    print(stack.pop()) # Output: 2
    print(stack.pop()) # Output: 1
    print(stack.is_empty()) # Output: True
```

Output:

```
3
3
False
2
1
True
```

Justification :

The Stack class is implemented using a Python list because lists provide efficient and built-in support for dynamic data storage. The `append()` and `pop()` operations work in constant time complexity ($O(1)$), making them ideal for stack operations that follow the LIFO (Last In, First Out) principle. Using a list also simplifies the implementation since it automatically handles memory management and resizing, reducing complexity in the code while maintaining efficiency and readability.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue: pass
```

Expected Output:

- FIFO-based queue class with `enqueue`, `dequeue`, `peek`, and `size` methods.

Prompt:

Write a Python Queue class using a list with `enqueue`, `dequeue`, `peek`, and `size` methods.

Code:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Peek from an empty queue")

    def size(self):
        return len(self.items)

    def is_empty(self):
        return len(self.items) == 0
```

```
# Example usage:
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek()) # Output: 1
print(queue.dequeue()) # Output: 1
print(queue.size()) # Output: 1
print(queue.is_empty()) # Output: False
print(queue.dequeue()) # Output: 2
print(queue.is_empty()) # Output: True
```

Output:

```
1
1
1
False
2
True
```

Justification :

The Queue class follows the FIFO (First In, First Out) principle, where the first element inserted is the first one to be removed. Using a Python list makes this implementation simple and readable. The append() method efficiently adds elements to the rear of the queue, while pop(0) removes elements from the front, maintaining proper queue behavior. This makes the implementation easy to understand for beginners and suitable for academic purposes.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node: pass
class LinkedList: pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt:

Write a Python Singly Linked List with insert and display methods.

Code:

```
#Write a Python Singly Linked List with insert and display methods.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()

# Example usage
sll = SinglyLinkedList()
sll.insert(10)
sll.insert(20)
sll.insert(30)
sll.display() # Output: 10 20 30
```

Output:

```
10 20 30
```

Justification :

The insert() method adds elements at the end of the list by traversing from the head node until it reaches the last node. This approach ensures that elements maintain their insertion order. Unlike arrays or lists, linked lists do not require continuous memory allocation, which makes them flexible for dynamic data storage. This design demonstrates how linked lists efficiently handle memory using node references.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST: pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

Prompt:

Write a Python Binary Search Tree class with insert and in_order_traversal methods.

Code:

```
#Write a Python Binary Search Tree class with insert and in_order_traversal methods.
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
    def __str__(self):
        return str(self.value)
class BinarySearchTree:
    def __init__(self):
        self.root = None
    def insert(self, value):
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert_recursive(self.root, value)
    def _insert_recursive(self, current_node, value):
        if value < current_node.value:
            if current_node.left is None:
                current_node.left = Node(value)
            else:
                self._insert_recursive(current_node.left, value)
```

```
    def _insert_recursive(self, current_node, value):
        if value < current_node.value:
            if current_node.left is None:
                current_node.left = Node(value)
            else:
                self._insert_recursive(current_node.left, value)
        else:
            if current_node.right is None:
                current_node.right = Node(value)
            else:
                self._insert_recursive(current_node.right, value)
    def in_order_traversal(self):
        return self._in_order_recursive(self.root)
    def _in_order_recursive(self, node):
        result = []
        if node is not None:
            result.extend(self._in_order_recursive(node.left))
            result.append(node.value)
            result.extend(self._in_order_recursive(node.right))
        return result
# Example usage:
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("In-order Traversal:", bst.in_order_traversal())
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
In-order Traversal: [2, 3, 4, 5, 6, 7, 8]
```

Justification :

The BST is implemented using recursion for both insertion and in-order traversal because trees naturally follow a recursive structure. Each subtree of a Binary Search Tree is itself a smaller BST, making recursion a logical and clean approach. This reduces code complexity and improves readability compared to iterative methods, especially for academic understanding.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable: pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

Write a Python HashTable class with insert, search, and delete methods using chaining for collision handling.

Code:

```
#Write a Python HashTable class with insert, search, and delete methods using chaining for collision handling.
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value) # Update existing key
                return
        self.table[index].append((key, value)) # Insert new key-value pair
```

```
    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v # Return the value associated with the key
        return None # Key not found

    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the key-value pair
                return True # Deletion successful
        return False # Key not found

# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
print(hash_table.search("name")) # Output: Alice
hash_table.delete("name")
print(hash_table.search("name")) # Output: None
```

Output:

```
Alice  
None
```

Justification :

The hash function uses Python's built-in `hash()` method combined with the modulo operator to ensure that the generated index stays within the table size. This guarantees uniform distribution of keys across buckets in most practical cases. Proper hashing improves average time complexity to $O(1)$ for insert, search, and delete operations, making the hash table highly efficient for large datasets.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph: pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt:

Write a Python Graph class using an adjacency list with methods to add vertices, add edges, and display connections.

Code:

```
#Write a Python Graph class using an adjacency list with methods to add vertices, add edges, and display connections.  
class Graph:  
    def __init__(self):  
        self.graph = {}  
  
    def add_vertex(self, vertex):  
        if vertex not in self.graph:  
            self.graph[vertex] = []  
  
    def add_edge(self, vertex1, vertex2):  
        if vertex1 in self.graph and vertex2 in self.graph:  
            self.graph[vertex1].append(vertex2)  
            self.graph[vertex2].append(vertex1)  
  
    def display_connections(self):  
        for vertex, edges in self.graph.items():  
            print(f"{vertex}: {' '.join(edges)}")  
  
# Example usage:  
g = Graph()  
g.add_vertex("A")  
g.add_vertex("B")  
g.add_vertex("C")  
g.add_edge("A", "B")  
g.add_edge("A", "C")  
g.display_connections()
```

Output:

```
A: B, C  
B: A  
C: A
```

Justification :

The graph is implemented using a Python dictionary where each key represents a vertex and its value is a list of connected vertices. This structure allows fast access to neighbors and makes adding vertices and edges straightforward. The dynamic nature of dictionaries ensures flexibility, as vertices can be added at runtime without predefined size limitations.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue: pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Prompt:

Write a Python PriorityQueue class using heapq with enqueue (priority), dequeue (highest priority), and display methods.

Code:

```
#Write a Python PriorityQueue class using heapq with enqueue (priority), dequeue (highest priority), and display methods.
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        raise IndexError("Dequeue from an empty priority queue")

    def is_empty(self):
        return len(self.elements) == 0
```

```
    def is_empty(self):
        return len(self.elements) == 0

    def display(self):
        print("Priority Queue:")
        for priority, item in sorted(self.elements):
            print(f"Item: {item}, Priority: {priority}")

# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", 2)
    pq.enqueue("Task 2", 1)
    pq.enqueue("Task 3", 3)

    pq.display()

    print("Dequeue:", pq.dequeue())
    print("Dequeue:", pq.dequeue())
    print("Dequeue:", pq.dequeue())
```

Output:

```
Priority Queue:
Item: Task 2, Priority: 1
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
Dequeue: Task 2
Dequeue: Task 1
Dequeue: Task 3
```

Justification :

The class design separates functionality into enqueue(), dequeue(), and display() methods, ensuring modularity and clarity. Error handling is included in the dequeue() method to prevent removal from an empty queue, improving reliability. This structured implementation makes the priority queue suitable for real-world applications such as task scheduling, process management, and event-driven simulations.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS: pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

Prompt:

Write a Python DequeDS class using collections.deque with methods to insert and remove from both ends, including docstrings.

Code:

```
s / shash / Downloads / AIAC / 11.py / ...
#Write a Python DequeDS class using collections.deque with methods to insert and remove from both ends, including docstrings.
from collections import deque
class DequeDS:
    """
    A class representing a double-ended queue (deque) data structure using collections.deque.

    Methods:
    - insert_front(item): Inserts an item at the front of the deque.
    - insert_rear(item): Inserts an item at the rear of the deque.
    - remove_front(): Removes and returns an item from the front of the deque.
    - remove_rear(): Removes and returns an item from the rear of the deque.
    - is_empty(): Checks if the deque is empty.
    - size(): Returns the number of items in the deque.
    """

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque."""
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.deque.append(item)
```

```
    def remove_front(self):
        """Remove and return an item from the front of the deque. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("Deque is empty")
        return self.deque.popleft()

    def remove_rear(self):
        """Remove and return an item from the rear of the deque. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("Deque is empty")
        return self.deque.pop()

    def is_empty(self):
        """Check if the deque is empty."""
        return len(self.deque) == 0

    def size(self):
        """Return the number of items in the deque."""
        return len(self.deque)

# Example usage:
if __name__ == "__main__":
    deque_ds = DequeDS()
    deque_ds.insert_rear(1)
    deque_ds.insert_rear(2)
    deque_ds.insert_front(0)
    print(deque_ds.remove_front()) # Output: 0
    print(deque_ds.remove_rear()) # Output: 2
    print(deque_ds.size())        # Output: 1
    print(deque_ds.is_empty())    # Output: False
```

Output:

```
0
2
1
False
```

Justification :

Separate methods such as `insert_front()`, `insert_rear()`, `remove_front()`, and `remove_rear()` clearly define operations at both ends of the deque. This improves readability and makes the behavior of the data structure explicit. Including proper error handling ensures that removing elements from an empty deque does not cause unexpected crashes, making the implementation reliable.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

Write a python code For each feature in a Campus Resource Management System (attendance, event registration, library borrowing, bus scheduling, cafeteria queue), choose the most appropriate data structure from Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, or Deque. Justify each choice in 2–3 sentences. Then implement one chosen feature as a Python program with comments and docstrings.

Code:

```
class Attendance:
    """
    A class to manage student attendance using a hash table (dictionary).
    """

    def __init__(self):
        """Initialize the attendance record as an empty dictionary."""
        self.attendance_record = {}

    def mark_attendance(self, student_id, status):
        """
        Mark the attendance of a student.

        Parameters:
        student_id (str): The unique ID of the student.
        status (str): The attendance status ('Present', 'Absent', 'Late').
        """
        self.attendance_record[student_id] = status

    def get_attendance(self, student_id):
        """
        Retrieve the attendance status of a student.

        Parameters:
        student_id (str): The unique ID of the student.

        Returns:
        str: The attendance status of the student or a message if not found.
        """
        return self.attendance_record.get(student_id, "Student not found")

    def display_attendance(self):
        """Display the entire attendance record."""
        for student_id, status in self.attendance_record.items():
            print(f"Student ID: {student_id}, Attendance Status: {status}")

# Example usage
attendance_system = Attendance()
attendance_system.mark_attendance("S001", "Present")
attendance_system.mark_attendance("S002", "Absent")
attendance_system.mark_attendance("S003", "Late")
print(attendance_system.get_attendance("S001")) # Output: Present
print(attendance_system.get_attendance("S004")) # Output: Student not found
attendance_system.display_attendance()
```

Output:

```
Present
Student not found
Student ID: S001, Attendance Status: Present
Student ID: S002, Attendance Status: Absent
Student ID: S003, Attendance Status: Late
```

Justification :

Each data structure is selected based on how closely its behavior matches the real-world requirement of the feature. For example, a Queue is chosen for cafeteria orders because it follows FIFO order, just like students waiting in line. Similarly, a Graph is ideal for bus scheduling since routes naturally form connected networks. Selecting the right structure ensures logical correctness and efficient implementation.

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

Generate a solution that maps each e-commerce feature to the most suitable data structure with 2–3 sentence justifications in a table format, and provide a fully functional Python program (with comments and docstrings) implementing one selected feature

Code:

```
Generate a solution that maps each e-commerce feature to the most suitable data structure with 2-3 sentence justifications in a table format, and provide a fully functional Python program implementing one selected feature.

# E-commerce Feature | Suitable Data Structure | Justification
#-----|-----|-----
# Product Catalog | Dictionary | A dictionary allows for efficient storage and retrieval of product information using unique product IDs as keys. It
# Shopping Cart | List | A list is suitable for a shopping cart as it can store multiple items in an ordered manner, allowing for easy addition and removal of items.

# Python program implementing the Shopping Cart feature
class ShoppingCart:
    """
    A class to represent a shopping cart in an e-commerce application.
    """

    def __init__(self):
        """Initialize an empty shopping cart."""
        self.cart = []

    def add_item(self, item):
        """
        Add an item to the shopping cart.

        Parameters:
        item (str): The name of the item to be added to the cart.
        """
        self.cart.append(item)
        print(f"Added {item} to the cart.")

    def remove_item(self, item):
        """
        Remove an item from the shopping cart.

        Parameters:
        item (str): The name of the item to be removed from the cart.
        """
        if item in self.cart:
            self.cart.remove(item)
            print(f"Removed {item} from the cart.")
        else:
            print(f"{item} not found in the cart.")

    def view_cart(self):
        """Display the contents of the shopping cart."""
        if self.cart:
            print("Shopping Cart Contents:")
            for idx, item in enumerate(self.cart, start=1):
                print(f"{idx}. {item}")
        else:
            print("Your shopping cart is empty.")

# Example usage
if __name__ == "__main__":
    cart = ShoppingCart()
    cart.add_item("Laptop")
    cart.add_item("Headphones")
    cart.view_cart()
    cart.remove_item("Laptop")
    cart.view_cart()
```

Output:

```
Added Laptop to the cart.  
Added Headphones to the cart.  
Shopping Cart Contents:  
1. Laptop  
2. Headphones  
Removed Laptop from the cart.  
Shopping Cart Contents:  
1. Headphones
```

Justification :

Each data structure is selected based on how closely its behavior matches the real-world requirement of the feature. For example, a Queue is ideal for order processing because it naturally follows FIFO order, ensuring fairness in handling customer orders. Similarly, a Graph accurately represents delivery routes since locations and connections form a network structure. Choosing structures that mirror real-life scenarios improves logical correctness.